A twenty-six-variable consistency equation

Positive integer unknowns; exact degree 12; a 124-operation evaluator.

The only unknowns are the twenty-six separately named positive-integer variables

$$J, F, \alpha, f, g, T, X, Y, \xi, \eta, \zeta, \upsilon, \gamma, \vartheta, \psi, \varphi, \epsilon, c, a, \ell, h, t, s, x, m, n \in \mathbb{Z}_{>0}.$$

All other symbols below are polynomial abbreviations:

$$\begin{gathered}
b = \beta J, \quad q = 1 + b, \quad C = q - F - \alpha, \quad U = Yq^2, \\
E = XY, \quad k = \eta + \zeta, \quad W = h - t, \\
H = 3 + 4a, \quad \Delta = H + a^2, \quad L = X + ac, \\
M = sb, \\
R = [\alpha + (q - F)b](q^2 - 1) + J(\mu + \omega q), \\
Z = \psi(\ell + k + \xi - kW) + Yb - Ws, \\
D = \varphi L + H(\varphi\gamma + x + \epsilon + 1 - k), \\
I = ng, \quad Q = \Delta(m - 1), \quad V = m(Tc - \epsilon) - c, \\
\mathcal{P} = Z^2 - \psi^2[1 + \ell(\ell + k)], \\
\mathcal{D} = D^2 - \varphi^2(1 + \Delta n), \\
\mathcal{Q} = \upsilon^2 - Q[\upsilon^2 - \vartheta] - 1, \\
\mathcal{R} = I^2 - Q, \quad \mathcal{E} = \vartheta - V^2, \\
\mathcal{B} = \psi - M, \quad \mathcal{C} = \varphi - U, \quad \mathcal{G} = \epsilon - R, \\
\mathcal{H} = c - \eta - Yk, \quad \mathcal{T} = a - E - Y, \\
\mathcal{L} = \ell - EYk, \quad \mathcal{W} = W - [C(X + Kq) - Fq], \\
\mathcal{X} = s - q^3, \quad \mathcal{Y} = x - Eq, \\
\mathcal{Z} = m - f^2, \quad \mathcal{N} = n - c^2.
\end{gathered}$$

The single Diophantine equation is

$$\begin{aligned}
&\mathcal{P}^2 + \mathcal{D}^2 + \mathcal{Q}^2 + \mathcal{R}^2 + \mathcal{E}^2 + \mathcal{B}^2 + \mathcal{C}^2 + \mathcal{G}^2 \\
&+\mathcal{H}^2 + \mathcal{T}^2 + \mathcal{L}^2 + \mathcal{W}^2 + \mathcal{X}^2 + \mathcal{Y}^2 + \mathcal{Z}^2 + \mathcal{N}^2 = 0.
\end{aligned}$$

The four fixed integers. Every index, set, sum, product and floor below defines a constant, not another unknown. Put

$$\begin{gathered}
\rho = 2^{47}, \quad B = \rho^{2011}, \quad \beta = B - 1, \\
G_m(x) = \sum_{j=0}^{m-1} x^j, \quad \Lambda = \rho^{1005}.
\end{gathered}$$

$$\begin{aligned}
\mu &= \beta - G_{951}(\rho) \\
&\quad - \rho^{960} G_{26}(\rho) - \rho^{1004}, \\
\omega &= \beta + 8G_{917}(\rho) - 4 + (2^{45} - 4)\Lambda \\
&\quad + \rho^{960} G_9(\rho) + \rho^{987} G_8(\rho) + \rho^{1004}, \\
K &= \rho^{18} + \rho^{1004} + \rho^{2020} \\
&\quad + 2\Lambda\Phi(1/\rho),
\end{aligned}$$

where

$$\begin{aligned}
\chi_i &= 1 + 2^{10}(1 + w_i + 2^8 n_i \\
&\quad + 2^{17} p_i + 2^{25} s_i) \quad (0 \le i \le 941), \\
\pi(0) = 0, \quad \pi(i) &= 1 + \sum_{j=1}^{941} \left\lfloor \frac{2^{44} - 1 + \chi_j - \chi_i}{2^{44}} \right\rfloor \quad (1 \le i \le 941).
\end{aligned}$$

$$\begin{aligned}
\Phi(x) = &\sum_{i=0}^{941} \chi_i x^{\pi(i)} \\
&+ 2^{10} x^{969}(x^9 + 2^{17}) G_8(2x) \\
&+ 2^{18} x^{942}(x^{18} + 2^{17}) G_9(2x).
\end{aligned}$$

The rational intermediate in K cancels exactly; all four results are positive integers.

The 942 indexed fixed tiles. Use the actions u_j, v_j, d_j decoded below. For $d = 0, 1$, let

$$\begin{gathered}
h(q, b) = 2q + b + 2, \quad \mathcal{S}_d = \{u_j : d_j = d,\ 0 \le j < 430\}, \\
r_{d,q} = 205 - 199d + |\mathcal{S}_d \cap \{0, \dots, q - 1\}|.
\end{gathered}$$

An interval ending at -1 is empty. The table defines t_i for $0 \le i \le 941$; all index ranges are fixed:

index i	tile $t_i = (w_i, n_i, p_i, s_i)$
0	$(2, 435, 3, 1)$
$j + 1$ $(j = 2q + b < 430)$	$((1 - d_j) r_{0,u_j}, h(q, b), d_j r_{1,u_j}, v_j)$
$433 - 2a + b$ $(a, b = 0, 1)$	$(0, 432a + b, 0, 432a + b)$
$\substack{423 + 2r_{d,q} + b \\ q \in \mathcal{S}_d,\ d,b = 0,1}$	$(dr_{d,q}, b, (1 - d) r_{d,q}, h(q, b))$
$923 + 2a + b$ $(a, b = 0, 1)$	$(0, 432a + b, 0, 438)$
927	$(2, 434, 2, 0)$
928	$(3, 436, 4, 4)$
929	$(4, 436, 4, 0)$
$496 + t$ $(t = 434, 435, 436)$	$(1, 438, 1, t)$
$933 + 3a + b$ $(a, b = 0, 1, 2)$	$(a^2, 437, b, 437)$

A closed decoder for the 430 actions. Put $D_b(A, k) = \left\lfloor A/b^k \right\rfloor \bmod b$, where $a \bmod b = a - b\lfloor a/b \rfloor$. For $0 \le q < 215$, $b = 0, 1$, $j = 2q + b$:

$$\begin{gathered}
\mathcal{N} = \mathcal{A} \bmod 2^{636}, \quad \mathcal{T} = \left\lfloor \mathcal{A}/2^{636} \right\rfloor, \\
\nu_j = D_2(\mathcal{N}, j), \quad \kappa_j = (\sigma_j + 2)^{1 - \nu_j} \quad (0 \le j < 430), \\
\sigma_j = \sum_{k=0}^{j-1} \nu_k, \quad \Pi_j = \prod_{k=0}^{j-1} \kappa_k \quad (0 \le j \le 430), \\
m_q = D_2(\mathcal{N}, 430 + q), \quad \tau_q = \sum_{k=0}^{q-1} m_k, \\
F_q = 14 + m_q(D_{16}(\lfloor \mathcal{T}/\Pi_{430} \rfloor, \tau_q) - 14), \\
u_j = (\sigma_{j+1} - (\lfloor \mathcal{T}/\Pi_j \rfloor \bmod \kappa_j)) \bmod 216, \\
v_j = D_2(F_q, 2b), \quad d_j = D_2(F_q, 2b + 1).
\end{gathered}$$

Empty sums/products are 0/1. Here $(v_0, d_0, u_0) = (1, 1, 1)$; the constructor tile uses this actual first target, including its state renaming.

The integer $\mathcal{A}$ in base 10. Its complete decimal expansion is the concatenation of these lines:

```
12889260410359561746736540743515845134244260640012702033212621
95398074180784058571518488282119861023093355389406939414042892
97455773992193784795705001618777025966369137954859764532832722
75411843410809038713502633204617333246733616940964578272360901
56186216508504953649192752154519268024509952082950238865468307
22999632482769253980274091217460707829023970817244790340704449
09085581271192071656170732004937337082171141082486384269072884
72778087419663417942892520175630649792329265834490253035577533
96449689808375788457488971017043689875433162972390177936123032
89025657757489526342497438772045714099661532481123050235898973
27443609903800541199351405985754278307227777584454508422717998
33411356440933534607489268032843 7
```

The fixed integer $\mathcal{A}$ above is written in base 10. Lean proves that the formalized equation has no positive solution exactly when the chosen arithmetized ZFC consistency statement holds, by a derivation in the specified intuitionistic PRA. The literal endpoint was independently replayed on a fixed source snapshot; some transitive dependencies remain inherited. The exact degree is certified by integer-polynomial calculations; the 124-operation count is for the unshifted circuit. Neither count is a minimality claim.

Exact 124-operation circuit

Each numbered line defines one gate over the integers. The inputs are the twenty-six unknowns and the four fixed integers above. The ASCII names `alpha, xi, eta, zeta, upsilon, gamma, vartheta, psi, varphi, epsilon, beta, mu, omega, ell, Delta` refer to their displayed counterparts; `g` is the unknown g. The last gate is zero exactly when the displayed equation holds. The circuit inputs `cNamed, aNamed, ellNamed` are c, a, ℓ; gate `WNamed` is the displayed W, while gate `W` is $C(X+Kq)-Fq$. The inputs `s, x, m, n` are the displayed s, x, m, n. Intermediate gate names below are local abbreviations, not additional unknowns.

```
1.  b = J * beta
2.  q = 1 + b
3.  q2 = q * q
4.  q3 = q * q2
5.  U = Y * q2
6.  qF = q - F
7.  C = qF - alpha
8.  E = X * Y
9.  k = eta + zeta
10.  kY = Y * k
11.  c = eta + kY
12.  c2 = cNamed * cNamed
13.  a = E + Y
14.  g15 = 4 * aNamed
15.  H = 3 + g15
16.  g17 = aNamed * aNamed
17.  Delta = H + g17
18.  g19 = aNamed * cNamed
19.  L = X + g19
20.  ell = E * kY
21.  M = s * b
22.  g25 = K * q
23.  g26 = X + g25
24.  g27 = C * g26
25.  g28 = F * q
26.  W = g27 - g28
27.  g30 = ellNamed + k
28.  g31 = g30 + xi
29.  WNamed = h - t
30.  root_skew = k * WNamed
31.  root_inner = g31 - root_skew
32.  root_scaled = psi * root_inner
33.  g33 = Y * b
34.  root_btq3 = WNamed * s
35.  root_sum = root_scaled + g33
36.  Z = root_sum - root_btq3
37.  g38 = Z * Z
38.  g39 = psi * psi
39.  g40 = ellNamed * g30
40.  g41 = 1 + g40
41.  g42 = g39 * g41
42.  A0 = g38 - g42
43.  g44 = qF * b
44.  rankseed = alpha + g44
45.  g46 = q2 - 1
46.  g47 = rankseed * g46
47.  g48 = omega * q
48.  g49 = mu + g48
49.  g50 = J * g49
50.  R = g47 + g50
51.  g52 = k - epsilon
52.  r0 = g52 - 1
53.  g54 = varphi * gamma
54.  g55 = E * q
55.  g56 = g54 + x
56.  shift = g56 - r0
57.  g58 = varphi * L
58.  g59 = H * shift
59.  D = g58 + g59
60.  g61 = D * D
61.  g62 = varphi * varphi
62.  g63 = Delta * n
63.  g64 = 1 + g63
64.  g65 = g62 * g64
65.  A1 = g61 - g65
66.  f2 = f * f
67.  rho = m - 1
68.  Q = Delta * rho
69.  I = n * g
70.  g71 = I * I
71.  g72 = g71 - Q
72.  g74 = T * cNamed
73.  g75 = g74 - epsilon
74.  g76 = m * g75
75.  V = g76 - cNamed
76.  y2 = upsilon * upsilon
77.  g80 = V * V
78.  g81 = y2 - vartheta
79.  g82 = Q * g81
80.  Fa = y2 - g82
81.  auxResidual = Fa - 1
82.  firstSquare = A0 * A0
83.  mainSquare = A1 * A1
84.  auxSquare = auxResidual * auxResidual
85.  strongSquare = g72 * g72
86.  firstMain = firstSquare + mainSquare
87.  firstMainAux = firstMain + auxSquare
88.  sumSquares = firstMainAux + strongSquare
89.  difference = vartheta - g80
90.  constraint = difference * difference
91.  result = sumSquares + constraint
92.  scaleDiff = psi - M
93.  scaleCheck = scaleDiff * scaleDiff
94.  final = result + scaleCheck
95.  mainDiff = varphi - U
96.  mainCheck = mainDiff * mainDiff
97.  result16 = final + mainCheck
98.  rankDiff = epsilon - R
99.  rankCheck = rankDiff * rankDiff
100.  result17 = result16 + rankCheck
101.  cDifference = cNamed - c
102.  cConstraint = cDifference * cDifference
103.  aDifference = aNamed - a
104.  aConstraint = aDifference * aDifference
105.  ellDifference = ellNamed - ell
106.  ellConstraint = ellDifference *
ellDifference
107.  wDifference = WNamed - W
108.  wConstraint = wDifference * wDifference
109.  addC = result17 + cConstraint
110.  addA = addC + aConstraint
111.  addEll = addA + ellConstraint
112.  finalWithNamedValues = addEll +
wConstraint
113.  cubeDifference = s - q3
114.  cubeConstraint = cubeDifference *
cubeDifference
115.  productDifference = x - g55
116.  productConstraint = productDifference *
productDifference
117.  fSquareDifference = m - f2
118.  fSquareConstraint = fSquareDifference *
fSquareDifference
119.  cSquareDifference = n - c2
120.  cSquareConstraint = cSquareDifference *
cSquareDifference
121.  addCube = finalWithNamedValues +
cubeConstraint
122.  addProduct = addCube + productConstraint
123.  addFSquare = addProduct +
fSquareConstraint
124.  addCSquare = addFSquare +
cSquareConstraint
```